AI Smart Navigation Assistant for Visually Impaired People Using Raspberry Pi

Smart Cane Environmental Awareness System

Final Project Report

Igor Xavier · **Fang Jialuo**

IoT Term Project

Repository: https://github.com/w1ggor/smart-nav-cane

June 26, 2026

Abstract

This report presents the design and implementation of a Smart Cane Environmental Awareness System: an offline, edge-AI assistive device for visually impaired users built entirely on a Raspberry Pi 4. The system combines a Time-of-Flight (ToF) depth camera, a USB webcam, and Bluetooth audio to provide three capabilities: (1) real-time obstacle detection and classification, (2) visual place recognition for trained indoor locations, and (3) voice-driven guided navigation with turn-by-turn audio instructions. The project deliberately combines classical computer vision (ORB feature matching) with deep learning (YOLOv8-nano for object classification, and a neural acoustic model inside Vosk for offline speech recognition), choosing each technique where it is most appropriate for a resource-constrained embedded platform. The system runs with no internet connection and no cloud dependency. We describe the architecture, hardware, software implementation, the engineering challenges encountered while integrating real hardware, and the current results, before outlining future work toward full indoor navigation.

Keywords: Raspberry Pi, assistive technology, edge AI, deep learning, object detection, speech recognition, computer vision, offline navigation.

Contents

1 Project Idea

1.1 Motivation

Visually impaired individuals face daily challenges navigating indoor environments safely and independently, particularly in unfamiliar or changing spaces. Commercial assistive devices that solve this problem comprehensively (e.g. devices using full SLAM-based mapping) are expensive and depend on specialized hardware such as inertial measurement units (IMUs) or LiDAR. Our goal was to build a low-cost, fully offline assistive device using commodity hardware that a student project can realistically build and demonstrate within an academic term.

1.2 Core Concept

The system is shaped like, and worn or carried like, a cane. It listens for a spoken destination, then guides the user there while continuously watching for obstacles and announcing them aloud through Bluetooth headphones. Three questions drive the entire design:

1. **What is in front of me?** — Obstacle detection (depth) and obstacle classification (deep learning).
2. **Where am I?** — Visual place recognition against previously trained locations.
3. **How do I get to my destination?** — Voice-driven guided navigation with turn-by-turn audio.

1.3 Scope Decision

The original project vision considered full indoor mapping using Visual SLAM (e.g. ORB-SLAM3) to build a 3D map and plan routes autonomously. After evaluating the available hardware — no IMU, no wheel odometry, a single Raspberry Pi 4 CPU — we determined that full SLAM was not achievable within the project timeline with acceptable reliability. We therefore redesigned the system around two achievable, complementary modes described in Section 2, while preserving the original vision explicitly as future work (Section 5).

1.4 Related Approaches

Commercial and research assistive navigation devices typically fall into two categories. **Sensor-fusion SLAM systems** (e.g. wearable LiDAR + IMU rigs) build a persistent 3D map and can route to unseen destinations, but require expensive sensors and significant onboard compute, putting them out of reach for a low-cost student prototype. **Simple sensory substitution devices** (e.g. ultrasonic canes that simply beep when close to an obstacle) are cheap and reliable but provide no semantic information — the user is warned *that* something is near, not *what* it is or *where they are*. This project deliberately sits between the two: it adds semantic understanding (object classification, place recognition, voice interaction) on top of low-cost depth sensing, without requiring the full sensor suite SLAM depends on.

1.5 Classical Computer Vision vs. Deep Learning in This Project

A central design decision was choosing, for each recognition task, whether classical computer vision or a learned model was more appropriate given the constraints of a single-board computer with no GPU. Table 1 summarizes this reasoning, which is revisited in more depth in Section 3.3.

Technique	Type	Training data needed	Used for
ORB + BFMatcher	Classical CV (hand-engineered features)	A 2-minute capture session per location/landmark, no labeling	Place and landmark recognition
YOLOv8-nano	Deep learning (CNN, pretrained)	None (uses public COCO weights)	Classifying generic obstacles (chair, person, etc.)
Vosk acoustic model	Deep learning (DNN-HMM, pretrained)	None (uses public English model)	Offline speech-to-text for destination input

Table 1: Recognition techniques used in the project, contrasted by type and data requirements.

2 System Design & Methodology

2.1 Architecture Overview

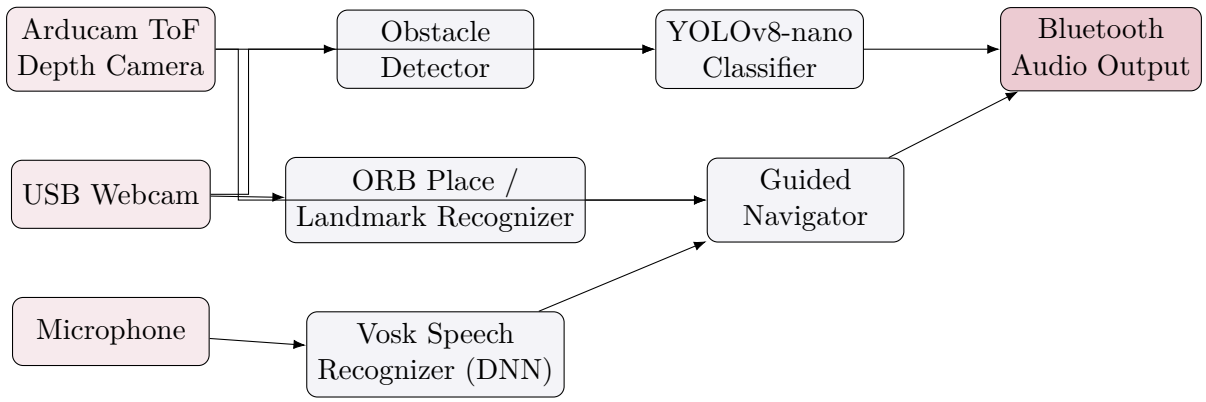

Figure 1: High-level data flow. Sensors (left) feed both classical CV and DL processing stages (center), which converge on the audio output (right).

The system runs two distinct operating modes built from the same underlying modules:

- **Awareness mode** — a continuous 2 Hz loop that detects obstacles and passively announces recognized locations, with no destination.
- **Guided navigation mode** — the user speaks a destination; the system gives turn-by-turn audio guidance, announces landmarks (e.g. doors) along the way, and confirms arrival.

2.2 Awareness Loop

Two concerns run every cycle: a cheap ToF depth threshold check (every cycle) and ORB-based place recognition (every third cycle, to reduce CPU load). Table 2 summarizes the loop.

Concern	Method	Frequency
Obstacle detection	Depth threshold, central zone of ToF frame	Every cycle (2 Hz)
Obstacle classification	YOLOv8-nano on webcam frame	Only when an obstacle is flagged
Location recognition	ORB feature matching vs. trained descriptors	Every 3rd cycle (∼1.5s)
Audio output	espeak-ng (TTS) via subprocess	On event

Table 2: Awareness loop concerns and their execution frequency.

2.3 Guided Navigation — Reactive Wall-Following

Without an IMU or wheel odometry, the system cannot plan a path to arbitrary coordinates the way SLAM-based navigation does. Instead, guided navigation reacts to the live ToF depth frame every cycle, split into three vertical zones (left, center, right):

- Center zone clear → *"Go straight."*
- Center zone blocked, one side clearer → *"Turn left" / "Turn right."*
- Both sides blocked → *"Path blocked. Please stop."*

This is combined with two independently loaded ORB recognizers: a *location* recognizer (checks for arrival at the destination) and a *landmark* recognizer (announces trained features such as doors when they are recognized close ahead). This approach works reliably for a previously walked, fixed route — a deliberate, honest trade-off documented further in Section 5.

2.4 Behavior Under Path Deviation

A natural question is what happens if the user does not follow the exact trained route. Because the system has no map and no odometry, it has, by design, no concept of being "on" or "off" a route — it only reacts to whatever the sensors currently see. This has two concrete consequences:

1. **Obstacle avoidance keeps working.** The wall-following logic (Listing 2) operates purely on the live ToF depth zones, so it continues to give locally correct "go straight" / "turn" guidance regardless of whether the user is on the trained path. Safety-relevant behavior degrades gracefully.

2. **Arrival and landmark recognition can silently fail.** Both depend on ORB matching the live webcam view against descriptors captured at training time. If the user's viewpoint differs enough from training — a different angle, distance, or lighting — recognition confidence can drop below threshold and the destination will not be detected as reached, with no error reported; the system simply keeps giving wall-following instructions indefinitely. There is no "you appear to be lost" fallback in the current version.

Section 4.2 quantifies how much viewpoint and lighting change ORB recognition can tolerate before this failure mode occurs, using a controlled desktop test.

2.5 Voice Destination Input

At the start of guided navigation, the system asks *"Where do you want to go?"* and transcribes the spoken response locally using Vosk, an offline speech-to-text engine. The transcribed text is matched against the labels of previously trained locations. No audio leaves the device — this preserves the project's fully-offline design goal even for voice interaction.

3 Hardware & Software Details

3.1 Hardware Components

Component	Interface	Role
Raspberry Pi 4 Model B (4GB)	—	Central compute; runs all processing locally
Arducam ToF Camera B0410	CSI	Depth sensing for obstacle detection (∼4m range, 240×180 @ 10fps)
Logitech C270 Webcam	USB (UVC)	Location/landmark recognition, YOLO obstacle classification
Bluetooth Headphones	A2DP	Spoken audio output
Bluetooth/USB Microphone	HSP/HFP or USB	Voice destination input

Table 3: Hardware components and their roles. Approximate total cost: €80.

Figure 2: Assembled prototype: Raspberry Pi 4, Arducam ToF camera (CSI), and USB webcam.

3.2 Software Architecture

The codebase is a modular Python package (`nav_assistant`) organized by concern:

Module	Responsibility
`sensors/`	Hardware abstraction for the webcam and ToF camera, with automatic V4L2 device detection by name
`mapping/`	Waypoint data model (SQLite + `.npy` ORB descriptors), distinguishing `location` vs. `landmark` kinds
`localization/`	ORB-based place/landmark recognition (classical CV)
`obstacle/`	ToF depth threshold detection
`perception/`	YOLOv8-nano obstacle classification (deep learning)
`speech/`	Vosk offline speech-to-text for destination input (deep learning)
`navigation/`	Reactive wall-following guided navigation
`audio/`	Text-to-speech (espeak-ng) and alert tones

Table 4: Software module structure.

3.3 AI / Machine Learning Components

The project guidelines require an explicit AI (ML/DL) component. Three recognition techniques are used deliberately, each chosen for where it performs best on constrained edge hardware:

1. **ORB feature matching** (classical computer vision, not learned) — used for visual place and landmark recognition. Chosen because it requires no training data beyond a short capture session, runs entirely on CPU, and is deterministic and debuggable.
2. **YOLOv8-nano** (convolutional neural network, deep learning) — a pretrained, COCO-trained object detector (80 classes, ∼6MB) used to classify what triggered a ToF obstacle alert (e.g. "chair", "person"), turning a generic alert into a specific one.
3. **Vosk speech recognition** (deep neural network acoustic model) — used for fully offline voice destination input, with no cloud speech API involved.

3.3.1 Dataset Sources

Each technique has a fundamentally different relationship to training data, summarized in Table 5.

Technique	Dataset source
ORB feature matching	**No external dataset.** Descriptors come entirely from the user's own environment, captured at runtime via the 2-minute training session (Table 1; implemented in `mapping/recorder.py`). Nothing is pretrained; nothing is shared between deployments.
YOLOv8-nano	**Microsoft COCO** (Common Objects in Context) — a public benchmark dataset of ∼330,000 images covering 80 object categories and ∼1.5 million labeled object instances. We use the publicly released, COCO-pretrained weights (`yolov8n.pt`) as distributed by Ultralytics, with no project-specific fine-tuning.
Vosk speech recognizer	`vosk-model-small-en-us-0.15` (40MB), a publicly released English acoustic and language model distributed by Alphacephei, the Vosk maintainers, trained on public English speech corpora. We use the released model directly, with no project-specific fine-tuning or additional training data.

Table 5: Dataset source for each AI/ML component used in the project.

Published accuracy benchmarks for the two pretrained models, as reported by their respective maintainers (not independently re-measured by us, since this would require either the original held-out test sets or hardware we no longer have access to):

- **YOLOv8-nano**: $\text{mAP}_{50\text{-}95} = 37.3$ on the COCO `val2017` benchmark, 3.2M parameters (Ultralytics official model documentation).
- **Vosk small English model**: word error rate (WER) of 9.85% on LibriSpeech `test-clean` and 10.38% on TEDLIUM (Vosk official model page).

3.3.2 Why YOLO Inference Is Gated, Not Continuous

Running a CNN on every frame would be too slow for real-time use on a Raspberry Pi 4 CPU (no GPU). The architecture instead gates the deep learning inference behind the cheap ToF depth threshold: `ObstacleDetector` runs every cycle as before, and YOLOv8-nano only runs on the corresponding webcam frame *after* an obstacle has already been flagged. This keeps the system real-time while still using a genuine deep learning model for recognition, illustrated in Listing 1.

Listing 1: Obstacle classification gated behind the cheap depth check (`perception/object_detector.py`).

```python
def classify(self, bgr_frame: np.ndarray) -> Optional[DetectionResult]:
    """Return the highest-confidence detection in the frame, or None."""
    if not self.is_available:
        return None

    results = self._model.predict(bgr_frame, verbose=False)[0]
    if results.boxes is None or len(results.boxes) == 0:
        return None

    best_idx = int(results.boxes.conf.argmax())
    confidence = float(results.boxes.conf[best_idx])
    if confidence < self._confidence_threshold:
        return None

    class_id = int(results.boxes.cls[best_idx])
    label = self._model.names[class_id]
    return DetectionResult(label=label, confidence=confidence)
```

3.3.3 Why ORB Is Used for Landmarks, Not YOLO

"Door" is not a class in the COCO dataset that YOLOv8-nano is pretrained on, and fine-tuning a custom YOLO class would require collecting and labeling a training dataset — impractical within the project timeline. Trained ORB landmarks (Listing 2) solve this with a two-minute capture session per landmark and no labeled dataset, at the cost of being less robust to lighting and viewpoint changes than a learned detector.

3.4 Guided Navigation Decision Flow

Figure 3 illustrates the per-cycle decision sequence implemented by `GuidedNavigator.evaluate()`: arrival is always checked first (highest priority, ends the session), then a landmark announcement is composed if relevant, and finally the wall-following turn/straight decision is computed from the three ToF zones.

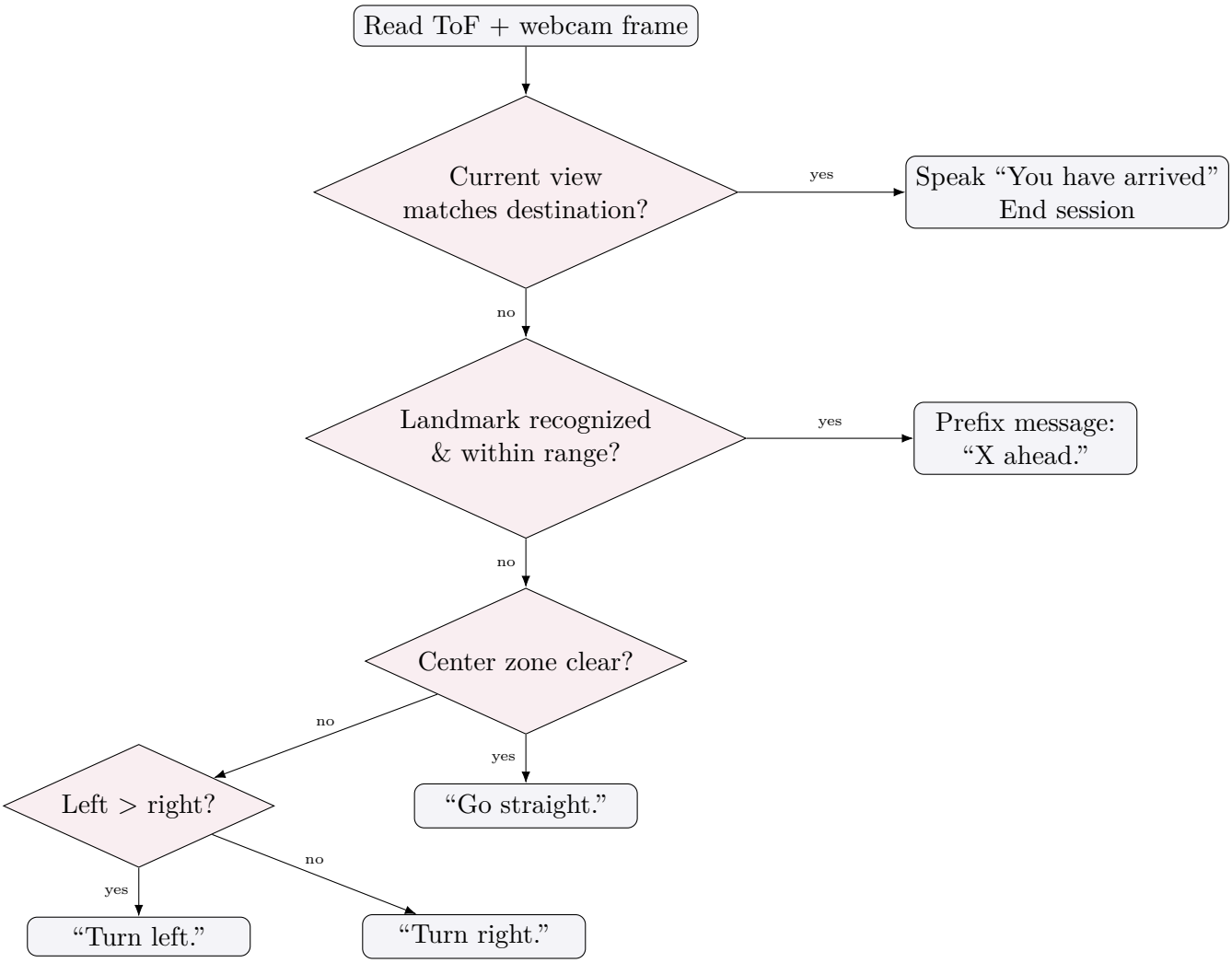

Figure 3: Decision flow for one guided-navigation cycle.

Listing 2: Reactive wall-following decision logic (`navigation/guided_navigator.py`).

```
def _wall_following(self, left: float, center: float, right: float):
    if center <= 0 or center > self._clear_distance:
        return GuidanceResult(NavCommand.STRAIGHT, "Go straight.")

    if left <= 0 and right <= 0:
        return GuidanceResult(NavCommand.STOP, "Path blocked. Please
            stop.")

    if left > right:
        return GuidanceResult(NavCommand.TURN_LEFT, "Turn left.")
    elif right > left:
        return GuidanceResult(NavCommand.TURN_RIGHT, "Turn right.")
    else:
        return GuidanceResult(NavCommand.STOP, "Path blocked. Please
            stop.")
```

3.5 Key Configuration Parameters

All thresholds are externalized to a single YAML configuration file rather than hardcoded, allowing the system to be re-tuned for a new environment without code changes.

Parameter	Default	Purpose
`obstacle.alert_threshold_m`	1.2 m	Distance below which an obstacle alert fires (awareness mode)
`navigation.clear_distance_m`	1.0 m	Center-zone depth above which the path is "clear" (guided navigation)
`navigation.emergency_threshold_m`	0.4 m	Tight safety override for something dangerously close
`navigation.landmark_distance_m`	1.5 m	Maximum distance for a landmark to be announced
`localization.min_good_matches`	15	Minimum ORB matches to accept a recognition
`localization.confidence_threshold`	0.6	Minimum confidence (0–1) to commit to a recognition
`perception.confidence_threshold`	0.5	Minimum YOLO detection confidence to report

Table 6: Selected tunable parameters from `config/default.yaml`.

4 Results & Contributions

4.1 Current Status

Phase	Status	Outcome
Sensor integration	Complete	Webcam and ToF camera validated on real hardware with automatic device detection
Location training	Complete	Multiple locations trained, each with 1000–5000 ORB descriptors
Awareness loop	Complete	Real-time obstacle alerts and location announcements running end-to-end
Guided navigation	Implemented	Voice input, wall-following, landmark announcement, arrival detection coded and unit-tested
DL obstacle classification	Implemented	YOLOv8-nano integrated, gated behind ToF threshold

Table 7: Project status by phase.

4.2 Accuracy & Robustness Validation

The Raspberry Pi hardware used during development was not available at submission time, so end-to-end accuracy could not be re-measured on the deployed platform. Two complementary, honestly-scoped sources of evidence are used instead.

4.2.1 Pretrained Model Benchmarks (YOLOv8-nano, Vosk)

For the two deep learning components, we report the official published benchmarks from each model's maintainer (Section 3.3.1) rather than re-deriving new numbers without the original test sets: YOLOv8-nano reaches $\text{mAP}_{50\text{-}95} = 37.3$ on COCO `val2017`; the Vosk

small English model reaches a 9.85% word error rate on LibriSpeech `test-clean`. Both are pretrained, unmodified, publicly verifiable figures.

4.2.2 Controlled ORB Robustness Test (This Work)

For the project's own classical-CV component, we ran a controlled desktop test: one reference photograph stands in for a "trained location," and the exact production matching code (`place_recognizer.py` — ORB with 500 features, BFMatcher, Lowe's ratio 0.75, `min_good_matches`=15, `confidence_threshold`=0.6) is used to match synthetically perturbed versions of that photo against the original. This characterizes ORB's sensitivity to the same kinds of change a user introduces by viewing a location from a different angle, while moving, or under different lighting — directly relevant to the path-deviation discussion in Section 2.4. It is a single-image desktop validation, not a statistically representative field accuracy figure for the deployed system. The test script and raw results are included in the project repository at `docs/report/validation/` for reproducibility.

Perturbation	**Result**	**Recognition threshold crossed**
In-plane rotation (0°–90°)	Good matches stayed above 375/500 at every angle tested	Never failed — ORB's built-in rotation invariance held
Gaussian blur (simulating motion blur)	Good matches fell from 500 (none) to 144 (heavy blur, kernel 15) to 79 (kernel 21)	Failed (confidence $<$ 0.6) at kernel $\geq$ 15
Brightness decrease (darker)	Good matches fell from 500 (unchanged) to 133 (-40) to 27 (-60)	Failed at -40 and below
Brightness increase (brighter)	Good matches remained $\geq$ 400 across all levels tested ($+20$ to $+60$)	Never failed

Table 8: ORB recognition robustness under synthetic perturbations of a single reference image, using the exact production matching parameters.

Interpretation. In-plane rotation alone is not a faithful proxy for the 3D viewpoint change a user introduces by approaching a location from a different angle (real viewpoint change also introduces perspective distortion, which this test does not capture) — so the true robustness to walking a different path is likely *lower* than the rotation row suggests. The blur and darkening results, however, are directly informative: they indicate degraded lighting or fast head/cane movement is more likely to cause a missed recognition than camera angle alone, which directly explains the failure mode described in Section 2.4.

4.3 Contributions

- A working, fully offline assistive prototype combining depth sensing, computer vision, and two deep learning models on a single Raspberry Pi 4.

- A deliberate, documented architecture that pairs classical CV (ORB) and deep learning (YOLO, Vosk) by matching each technique to where it performs best, rather than defaulting to one approach.
- A robust hardware abstraction layer that automatically detects camera devices by name, solving a real instability (USB device indices changing across reboots) discovered during development.
- 27 automated unit tests covering the data model, sensor processing, navigation decision logic, and DL integration's graceful degradation.

4.4 Engineering Challenges

This section focuses on the four challenges that most shaped the project's architecture and process, rather than a full bug list (several smaller hardware integration fixes are documented in the project's change history).

4.4.1 Challenge 1: Full SLAM Was Infeasible — Redesigning Around It

The original project plan was full Visual SLAM (e.g. ORB-SLAM3): build a 3D map in real time and plan routes to arbitrary destinations. Partway into development it became clear this was not achievable with the available hardware and timeline — there is no IMU, no wheel odometry, and a single CPU-only Raspberry Pi 4, all of which SLAM implementations typically depend on for motion estimation between frames. Rather than attempting it anyway and shipping an unreliable map, we treated this as a scope decision, not a failure: the system was redesigned around two narrower but achievable modes — passive awareness and reactive wall-following guided navigation (Sections 2–2.4) — while keeping SLAM as an explicit, documented future-work item (Section 5) rather than silently dropping it.

4.4.2 Challenge 2: Training Data Evolved Through Three Iterations

Visual place recognition's reliability turned out to depend heavily on how training data was collected, which we only discovered by testing the simplest approach first:

1. *Single frame.* The first implementation captured one webcam frame per location. Recognition was unreliable — a single viewpoint does not capture enough visual diversity for ORB to match later, naturally varying views.
2. *Short burst.* Capturing 5 frames in quick succession improved things slightly, but frames taken within roughly a second of each other are still nearly the same viewpoint, so diversity barely improved.
3. *Continuous two-minute capture (final).* The training tool now samples one frame every 3 seconds for 2 minutes while the user walks around the space, yielding ∼40 frames and up to 5,000 descriptors per location (randomly subsampled if more are collected, to bound matching cost). This is also the data source documented in Table 5 for ORB's "no external dataset" design.

This is a direct, practical illustration of why dataset *construction methodology* matters as much as dataset *size* for a classical CV technique with no learned generalization.

4.4.3 Challenge 3: A Thread-Safety Bug That Only Appeared on a Different Platform

While preparing this submission without access to the Raspberry Pi, we built a desktop demo tool (running on Windows) and discovered a real bug invisible on the original target platform: the `pyttsx3` text-to-speech fallback shares one engine instance between a background "speak" thread and the main thread's "alert" call. On Windows, `pyttsx3`'s SAPI5 driver is COM-based and apartment-threaded, meaning a single engine instance may only safely be called from the thread that created it — calling it from a second thread silently hangs with no exception. The fix creates a fresh, short-lived engine inside every call, scoped entirely to whichever thread is using it. This had no effect on the Raspberry Pi, which uses the `espeak-ng` subprocess backend instead (each call is an independent process, inherently thread-safe) — but would have caused a hard-to-diagnose hang for any future Windows or macOS deployment.

4.4.4 Challenge 4: Gating Deep Learning Inference Behind a Cheap Check

Running YOLOv8-nano on every frame would be too slow for real-time use on a CPU-only Raspberry Pi 4. The solution (Section 3.3) runs the cheap ToF depth threshold every cycle as before, and only invokes the CNN on the webcam frame after an obstacle has already been flagged — paying the deep learning inference cost only when it is actually useful.

5 Future Work

The features below were part of the original project vision and were consciously deferred to keep the current scope achievable within the term:

- **Visual SLAM** — building a 3D map of the environment in real time (e.g. ORB-SLAM3) instead of reactive wall-following, enabling navigation to destinations not directly visible.
- **Graph-based route planning** — the data model already stores directed edges between locations (`WaypointEdge`); a Dijkstra-based planner could route across multiple reactive legs.
- **IMU integration** — dead-reckoning between locations would allow distance and orientation tracking independent of visual recognition.
- **Fine-tuned object detection** — training a custom YOLO model on assistive-technology-relevant classes (doors, stairs, curbs) instead of relying on ORB landmarks and generic COCO classes.
- **Multi-environment support** — automatic environment selection when the device is carried between different trained spaces.

6 Conclusion

This project demonstrates that a meaningful assistive navigation system can be built entirely offline on commodity Raspberry Pi hardware by combining classical computer vision and deep learning where each is most effective. Rather than attempting full SLAM-based navigation with insufficient sensors and time, the team redesigned the scope around two achievable, genuinely useful modes — environmental awareness and reactive guided navigation — while explicitly preserving the original ambitions as a documented roadmap. The

resulting system detects and classifies obstacles, recognizes trained locations, accepts voice commands, and gives turn-by-turn audio guidance, all without an internet connection.